AXIS

Complete Product & Engineering Specification

Version 1.0 | Coding-agent handoff | Implementation not started

Everything in sync.

AXIS — Complete Product and Engineering Specification

Version 1.0 | Documentation pack | Implementation not started

> This compilation is for convenient review. The individual Markdown files in the ZIP remain the canonical modular documents.

AXIS — Master Context

Version: 1.0 | Status: Approved product direction, engineering details partly open

Owner: Product owner / single initial user

Product name: AXIS

Tagline: “Everything in sync.”

1. Product in one paragraph

AXIS is a personal AI operating system and autonomous digital employee that reduces the user's cognitive load. It captures commitments, remembers useful context, organizes projects, proposes realistic plans, sends durable reminders, coordinates connected apps, and prepares actions for approval. The user should not have to mentally track every task, deadline, follow-up, or personal goal. AXIS can be proactive about surfacing what needs attention, but it must not independently perform consequential external actions without explicit approval.

2. Product principles

1. **Trust before cleverness.** Reliable reminders and accurate memory matter more than impressive demos.
2. **One source of truth.** Web dashboard, WhatsApp, scheduler, and integrations must read/write the same canonical records.
3. **Proactive, not presumptuous.** AXIS may suggest, plan, draft, and notify. It waits for approval before external-state-changing actions.
4. **Persistent by design.** A task or reminder must survive a chat reset, model outage, worker restart, and deployment.
5. **Transparent uncertainty.** Label confirmed facts, assumptions, and suggestions distinctly.
6. **Low cognitive load.** Concise briefings, sensible defaults, clear next actions, minimal unnecessary questions.
7. **Privacy and control.** User can inspect, correct, archive, export where feasible, and delete memory. Ask before saving sensitive personal facts.
8. **Recoverable.** Backups, idempotency, delivery tracking, and tested restore procedures are part of the product—not future polish.
9. **Build in slices.** Ship a trustworthy core before expanding into voice, broad browser automation, or multi-user SaaS.
10. **Future-ready, not prematurely complex.** Design for user/tenant isolation but begin as a single-user deployment.

3. Initial user and context

Initial release is for one user. The user's current workload can include CAT preparation, college/dissertation work, client delivery, multiple projects/business initiatives, personal-brand content, fitness, and routine commitments. This context is illustrative; it must be editable and must not be hard-coded into business logic.

4. Desired user experience

- AXIS remembers commitments and surfaces them at the right time.
- A concise morning briefing is generated every morning.
- A rolling weekly plan is broken into realistic daily time blocks, respecting fixed commitments, focus blocks, rest, and buffers.
- Missed tasks are detected and a revised plan is proposed for approval.

- A reminder created without a time gets a suggested time based on deadline and importance; AXIS asks for confirmation before scheduling it.
- WhatsApp is a first-class conversational interface; the web dashboard uses the same state.
- If WhatsApp delivery is not confirmed within a configured timeout/retry policy, email fallback is attempted for that reminder. Do not send duplicates unnecessarily.
- Calendar integration begins read-only. Calendar changes require explicit approval.
- Voice notes and speech responses may follow the core MVP; live voice calls are deferred.
- The assistant has a natural, intelligent, concise, slightly witty personality. Tone must never obscure dates, statuses, risks, or confirmations.

5. Autonomy model

Allowed without external-action approval

- Answer questions using accessible records.
- Search and summarize stored memory.
- Propose plans and schedules.
- Draft messages, emails, calendar events, and other actions.
- Create or update internal AXIS records when the user has clearly instructed it to do so and the change is not an external side effect. For ambiguous instructions, ask first.
- Send configured reminders/briefings through the user's explicitly connected and consented notification channels, under the notification policy.

Requires explicit approval

- Sending a message/email to another person.
- Creating, modifying, or deleting events in an external calendar.
- Publishing content, submitting forms, making purchases, transferring money, deleting external records, or taking actions that represent the user.
- Any other action that changes state outside AXIS, unless a narrow, explicitly approved automation policy is added later.

Additional high-risk gate

High-risk actions require both explicit action approval and fresh web authentication. The exact list of high-risk actions and freshness window are open decisions. Until defined, use a conservative classification and do not execute uncertain high-risk actions.

6. Memory model

AXIS has layered global and project-specific memory. Memory is not the same thing as a task, event, or reminder.

- **Global memory:** durable preferences, recurring routines, stable context.
- **Project memory:** goals, decisions, stakeholders, deliverables, constraints, and history scoped to a project.
- **Task records:** actionable work with status, priority, due date, dependencies, and ownership.
- **Reminder records:** scheduled notification intent and delivery state.
- **Conversation history:** useful for context but not authoritative for obligations.
- **Audit events:** append-only record of security-sensitive actions and outcomes.

Remember routine information by default. Ask before saving sensitive/personal facts. Each memory item should support provenance, confidence/status, timestamps, scope, and user correction/deletion. Distinguish confirmed, assumed, and suggested information. Archive inactive project memories without making them unsearchable.

7. MVP boundary

Must ship in MVP

- Basic authenticated single-user web dashboard.
- Global/project memory capture, search, inspect, correct, archive, and delete.
- Task and commitment capture, status, priority, due date, project linkage, and confirmation flow for ambiguous input.
- Durable reminders stored in the database and processed by a separate worker.
- WhatsApp Sandbox integration for inbound messages and outbound reminders/approvals.
- Email fallback for any reminder whose WhatsApp delivery is not confirmed, using delivery state, timeout, bounded retries, and deduplication.
- Approval queue and server-enforced approval gate.
- Basic daily briefing and simple daily/weekly plan proposals.
- Audit trail, structured error states, monitoring hooks, backup automation, and restore-test procedure.
- Automated tests for the release blockers in `docs/09-test-plan-and-acceptance.md`.

Later, not required for initial release

- Production WhatsApp sender and broader account onboarding.
- Live WhatsApp calls or phone calling.
- Full voice assistant; initial follow-on is voice-note transcription and speech responses.
- Broad browser automation and autonomous execution across third-party apps.
- Deep CRM/pipeline, CAT/fitness analytics, and rich integrations beyond the initial need.
- Multi-user commercial SaaS launch, billing, tenant administration, and complex role management.
- Dedicated vector database or separate distributed queue without demonstrated need.

8. Success metrics

Initial success is trust: the user relies on AXIS to remember commitments and surface important reminders.

Measure:

- Reminder delivery success and time-to-delivery by channel.
- Number of reminders with unknown/unresolved delivery outcome.
- Duplicate notification/action rate.
- Missed/late reminder incidents.
- Approval-gate bypass test results (must be zero).
- Task capture completeness and user correction rate.
- Memory retrieval relevance and correction/deletion success.
- Backup completion and restore-test success.
- User-reported usefulness of morning briefing and weekly plan.

Do not optimize for message volume or time spent in the app.

9. Reliability and recovery defaults

- Approval expiry: 24 hours.
- High-risk action: explicit approval plus fresh web authentication.
- Backup retention: 7 daily backups.
- Restore test: monthly.
- Recovery Point Objective (RPO) target: 24 hours.
- Recovery Time Objective (RTO) target: 4 hours.

- These are engineering targets, not guarantees; validate the chosen host/database plan.
- Bounded retries with exponential backoff and jitter where suitable.
- Reconcile uncertain provider timeouts before retrying consequential actions.
- Prevent duplicate effects with idempotency keys and provider message IDs.
- If WhatsApp delivery is not confirmed, email fallback is required by policy.
- Model provider failure must not stop reminders from running.

10. Product decisions still open

Do not silently lock these:

- Final hosting provider and exact free-tier limits.
- Exact model provider(s), pricing limits, and fallback order.
- Production WhatsApp provider/sender and delivery-status capabilities.
- Email provider and sending domain.
- Exact user timezone setting UX; persist IANA timezone, default initially from explicit user setup.
- High-risk action taxonomy and authentication freshness window.
- Approval revocation behavior after execution starts.
- Email fallback timeout and number/timing of retries.
- Quiet hours and urgent exception categories; user-configurable.
- Whether pgvector is needed after testing PostgreSQL full-text search.
- Exact calendar provider and OAuth scopes.
- Long-term multi-user onboarding and billing scope.

11. Language and status conventions

UI and logs must distinguish:

- **Saved** — persisted successfully.
- **Scheduled** — reminder/job exists and has a due time.
- **Sent** — provider accepted/sent the notification; not necessarily read.
- **Delivered** — provider confirmed delivery where available.
- **Unknown** — outcome cannot be confirmed.
- **Failed** — confirmed failure or retries exhausted.
- **Needs approval** — prepared but not authorized to execute.
- **Completed** — verified outcome, not merely a model claim.

Never say “delivered” if the provider only confirms “sent.”

AXIS — Product Requirements Document (PRD)

Version: 1.0 | Status: MVP scope baseline

1. Problem statement

The user has many concurrent commitments and does not want to mentally track tasks, deadlines, project details, reminders, and follow-ups. Existing chat assistants can answer questions but are not a trustworthy system of record for obligations. AXIS must combine useful memory, structured work records, proactive planning, durable reminders, and approval-gated actions.

2. Goals

- Capture commitments quickly from web and WhatsApp.
- Retrieve relevant global and project context.
- Maintain reliable task and reminder state outside model context.
- Generate a concise daily briefing and rolling weekly plan.
- Detect overdue/missed tasks and propose a revised schedule.
- Notify through WhatsApp with email fallback when delivery is unconfirmed.
- Prepare external actions and require approval before execution.
- Provide enough visibility to inspect what AXIS remembers and what it has done.

3. Non-goals for MVP

- Fully autonomous external execution.
- Live calling, full duplex voice, or wake-word assistant.
- Guaranteed zero downtime or guaranteed free hosting.
- Production multi-tenant SaaS, billing, or team collaboration.
- Replacing a full project-management or CRM suite.
- Storing payment credentials or performing financial transactions.
- Unrestricted browser access or arbitrary code execution by model output.

4. Personas and primary jobs

Primary user

A single individual managing personal goals, studies, projects, clients, and routines.

Jobs to be done

1. “Remember this and help me find it later.”
2. “Turn this commitment into a task and remind me at the right time.”
3. “Tell me what matters today and propose a realistic plan.”
4. “Notice when I have fallen behind and help me recover.”
5. “Prepare this external action, show me exactly what will happen, and wait for my approval.”
6. “Tell me whether a reminder or action actually succeeded.”

5. Functional requirements

Priority labels: P0 release blocker, P1 core MVP, P2 follow-on.

FR-01: Identity and access (P0)

- Support one owner account initially.
- Protect dashboard and APIs behind authentication.
- Keep an architecture seam for future user/tenant IDs.
- Do not expose unauthenticated memory, tasks, reminders, approvals, logs, or integration settings.
- Authentication provider is an open implementation decision; choose a secure, maintained option and document it before wiring real accounts.

Acceptance: unauthenticated API requests are denied; all user-owned records are scoped to the authenticated owner.

FR-02: Quick capture (P1)

- Accept natural-language input in web chat and WhatsApp.
- Extract candidate task, commitment, project, date/time, recurrence, and importance.
- Validate model output against a schema.
- Ask clarifying questions for materially ambiguous instructions.
- Do not silently create an external action from an ambiguous request.
- If a reminder time is missing, propose a time based on deadline/importance and ask the user to confirm.

Acceptance: invalid or incomplete model output never creates malformed records; ambiguous date/time produces a clarification or proposal.

FR-03: Memory (P0)

- Support global and project-scoped memory.
- Each item includes content, type, scope, source/provenance, confidence/status, created/updated timestamps, and optional expiry/archive status.
- Remember routine information by default; ask before saving sensitive/personal facts.
- Allow inspect, edit/correct, archive, delete, and search.
- Mark records as confirmed, assumed, or suggested.
- Inactive project memory can be archived but remains searchable.
- Avoid saving credentials or secrets as memory.

Acceptance: users can find and correct an item; deleted items are not returned by ordinary retrieval; project-scoped search does not leak into another user's scope when multi-user support is added.

FR-04: Tasks and commitments (P0)

- Create, view, edit, complete, cancel, defer, and archive tasks.
- Fields: title, description, status, priority, due date/time, timezone, estimated duration, project, dependencies, source, created/updated timestamps.
- Use explicit status values, not ambiguous free text.
- Track task history for meaningful status changes.
- Completion of recurring tasks generates the next occurrence according to its recurrence rule.
- Missed tasks are detected and a reschedule proposal is prepared, not silently applied.

Acceptance: task state survives restart; completing a recurring task produces exactly one next occurrence.

FR-05: Durable reminders (P0)

- Reminders are database records with due time, timezone, linked entity, channel policy, status, delivery attempts, and next retry.
- Scheduler/worker is independent of chat and model providers.
- Support reminders at creation, before deadline, on due date, and after overdue; exact offsets are configurable and must be surfaced to the user when creating the reminder.
- Every notification attempt has a traceable result.
- WhatsApp delivery not confirmed within configured policy triggers email fallback.
- Deduplicate across retries and webhook replays.
- Quiet hours and focus blocks are respected, with user-configurable urgent exceptions.

Acceptance: restart/model outage does not erase reminders; simulated provider timeout results in a tracked unknown state and appropriate retry/fallback.

FR-06: Approvals and external actions (P0)

- Any external-state-changing action must be represented as an action proposal with a unique ID, immutable payload, destination, risk level, expiry, and status.
- Approval can be given through authenticated web UI or a verified WhatsApp interaction linked to the user.
- Backend validates the approval and action payload before execution.
- Approval expires after 24 hours.
- High-risk actions require explicit approval plus fresh web authentication.
- Material payload/destination change invalidates prior approval.
- Approval is consumed once; concurrent/replayed requests cannot execute twice.
- Record proposal, approval/rejection/expiry, execution attempt, and outcome in the audit trail.
- If execution outcome is uncertain, reconcile before retrying.

Acceptance: attempts to execute without valid approval fail server-side even if the client calls the endpoint directly.

FR-07: Daily briefing and planning (P1)

- Produce a concise morning briefing with today's fixed commitments, top priorities, deadlines, reminders needing attention, and pending approvals.
- Maintain a rolling weekly plan with time blocks.
- Fixed commitments first; then priorities; then flexible tasks, rest, and buffers.
- Consider urgency, importance, effort, dependencies, available time, and long-term goals.
- The plan is a proposal. User approval is required before it changes external calendars or applies consequential rescheduling.
- Clearly label estimated duration and uncertain assumptions.

Acceptance: planner does not schedule two tasks into the same time block or over a fixed commitment; it leaves buffers and flags unschedulable work.

FR-08: Notifications and inbox (P0)

- Show pending approvals, failed/unknown reminders, upcoming deadlines, and important system alerts in a central inbox.
- Send configured reminders and briefings through consented channels.
- WhatsApp is primary for MVP; email is fallback if WhatsApp delivery is unconfirmed.
- Delivery status is explicit: queued, sent, delivered (if confirmed), failed, unknown, fallback queued/sent, or cancelled.
- Do not confuse provider acceptance with confirmed delivery.

FR-09: Integrations (P1/P2)

- MVP: WhatsApp Sandbox; email fallback; optional read-only Google Calendar after core reminder reliability is proven.
- Calendar is read-only initially. Calendar writes require approval and are out of initial scope unless explicitly enabled.
- Use least-privilege scopes, revocable credentials, provider adapters, and clear connection status.
- Integration errors should not corrupt core records.

FR-10: Dashboard and chat (P1)

- Responsive web dashboard, desktop and mobile.
- Dark mode default; optional light mode.
- Hybrid command center + persistent assistant panel.
- Show today's agenda, priority tasks, deadlines, pending approvals, recent memory/conversations, project status, notification health, and quick capture.
- Mobile defaults to chat-first with easy dashboard access.
- Keep UX concise; make status and next action obvious.

FR-11: Observability and recovery (P0)

- Structured logs, correlation IDs, audit events, health checks, and metrics for reminder delivery, retries, approval outcomes, and worker health.
- Daily automated database backups, seven daily copies retained.
- Monthly isolated restore test.
- Target RPO ≤24 hours and RTO ≤4 hours, subject to hosting/provider validation.
- Reconcile reminder schedules and delivery attempts after recovery.

6. Core user flows

Flow A: Save a memory

1. User shares a fact or preference.
2. AXIS classifies it as routine or potentially sensitive.
3. Routine facts may be saved by default; sensitive/personal facts require confirmation.
4. AXIS reports what was saved and how to inspect/edit/delete it.

Flow B: Create a task and reminder

1. User asks to remember a task.
2. AXIS extracts fields and checks ambiguity.
3. If date/time is missing, propose a reminder time and ask for confirmation.
4. Persist task and reminder transactionally where appropriate.
5. Scheduler claims due reminder and records an attempt.
6. WhatsApp sends; delivery status is tracked.
7. If delivery is unconfirmed after policy timeout/retries, queue email fallback.
8. Surface final status honestly.

Flow C: Approve external action

1. AXIS prepares a draft and immutable payload.
2. User sees destination, exact content/effect, risk, and expiry.
3. User approves via authenticated dashboard or verified WhatsApp flow.
4. Backend checks ownership, status, expiry, payload hash, risk gate, and idempotency.
5. Execute once; persist result and audit event.
6. If outcome is unknown, reconcile before retrying.

Flow D: Recover missed tasks

1. Worker detects overdue tasks according to policy.
2. Planner considers constraints and remaining capacity.
3. AXIS presents a proposed revised plan.
4. User approves or edits it.
5. AXIS updates internal plan records; external calendar changes remain approval-gated.

7. MVP release gates

Release is blocked by any of:

- Critical reminder can be lost silently.
- Approval gate can be bypassed.
- Duplicate consequential action can occur in retry/replay/concurrency tests.

- Failed recovery/restore test.
- Secret exposure in repository, logs, client bundles, or docs.
- User-owned data can be accessed across identities.
- Reminder fallback policy is untested.
- No documented runbook for worker/provider failure.

8. Analytics events

Use privacy-minimizing event names and avoid raw sensitive content:

- `task_created`, `task_completed`, `task_deferred`
- `memory_saved`, `memory_corrected`, `memory_deleted`
- `reminder_scheduled`, `reminder_attempted`, `reminder_delivered`, `reminder_failed`, `reminder_fallback_triggered`
- `approval_requested`, `approval_accepted`, `approval_rejected`, `approval_expired`
- `action_execution_succeeded`, `action_execution_failed`, `action_execution_unknown`
- `backup_completed`, `restore_test_completed`

Analytics must not store message bodies, memory text, access tokens, or unnecessary personal details.

AXIS — Technical Architecture

Version: 1.0 | Approach: Modular monolith + independent worker

1. Recommended architecture

Start with a small modular monolith. Keep modules and integration adapters separate in code, but do not split into multiple deployable services until a real scaling or isolation need appears.

```
Web app (React)
   | HTTPS / authenticated API
   v
Node.js API (Express, TypeScript)
   |-- Auth / user scope
   |-- Assistant orchestration / model adapters
   |-- Memory / search
   |-- Tasks / projects / planning
   |-- Reminders / delivery policy
   |-- Approvals / action execution gate
   |-- Integrations / webhooks
   |-- Audit / observability
   |
   v
PostgreSQL (source of truth)
   ^
   | claims due work, writes delivery outcomes
Worker / scheduler (separate process)
   |-- due reminders
   |-- retries and email fallback
   |-- recurring tasks
   |-- morning briefing jobs
   |-- reconciliation / health checks
   |
   +--> WhatsApp provider adapter (Sandbox first)
   +--> Email provider adapter
   +--> Optional n8n workflows for bounded integrations
   +--> Optional model provider adapters
```

The web app and worker may live in the same repository and share domain packages. They must be independently runnable processes so model or web API outages do not stop scheduled work.

2. Proposed stack

These are recommended implementation defaults, not a mandate to provision specific paid services.

- Frontend: React + TypeScript + Vite.
- UI: Tailwind CSS or another consistent accessible component system.
- Backend: Node.js + TypeScript + Express.
- Database: PostgreSQL.
- Data access: a typed ORM/query builder with migrations (choose one and document why; do not mix ORMs).
- Validation: runtime schemas for API payloads and model outputs.
- Worker: Node process using database-backed job claiming and retry schedules.
- Integrations: provider adapters; n8n can orchestrate selected workflows but must not become the source of truth.
- WhatsApp: Twilio Sandbox initially, subject to current account and region constraints.
- Email: provider adapter, provider choice OPEN.
- AI: configurable model-provider interface with bounded fallback.
- Search: PostgreSQL full-text search first; evaluate pgvector only when retrieval evaluation demonstrates value.
- Tests: unit, integration, API contract, browser/e2e, security and recovery tests.
- Deployment: free-tier-first, managed PostgreSQL if available, with limits documented. Validate backups, worker sleep behavior, cron guarantees, and outbound messaging before selecting a host.

3. Repository layout

```
axis/
■■■ AGENTS.md
■■■ README.md
■■■ .env.example
■■■ package.json
■■■ apps/
■   ■■■ web/
■■■ services/
■   ■■■ api/
■   ■■■ worker/
■■■ packages/
■   ■■■ shared/
■   ■■■ config/
■   ■■■ domain/
■■■ database/
■   ■■■ migrations/
■   ■■■ seeds/
■   ■■■ scripts/
■■■ workflows/
■   ■■■ n8n/
■■■ tests/
■   ■■■ integration/
■   ■■■ e2e/
■   ■■■ security/
■   ■■■ recovery/
■■■ docs/
```

Keep initial structure pragmatic. If the agent selects a monorepo tool, document the choice and preserve the boundaries above.

4. Domain boundaries

- **Identity:** user identity, sessions, authorization scope.
- **Memory:** facts/preferences, provenance, confidence, search, correction/deletion.
- **Projects:** project metadata and project-scoped context.
- **Tasks:** actionable work, recurrence, dependencies, status history.
- **Planning:** time constraints, time blocks, proposals, plan acceptance.
- **Reminders:** due times, scheduling, delivery policy and status.
- **Notifications:** provider send attempts, webhook status, email fallback.
- **Approvals:** immutable action proposal, approval lifecycle, risk gate.
- **Actions:** adapters that execute explicitly approved external operations.
- **Integrations:** credentials references, connection health, inbound webhooks.

- **Audit:** append-only security and action events.
- **Assistant:** language understanding, retrieval, tool selection, response synthesis. It coordinates domain APIs; it does not bypass them.

5. Source-of-truth rules

PostgreSQL is authoritative for tasks, reminders, plans, approvals, delivery attempts, integration metadata, and audit events. Chat transcripts and model context are not authoritative. External providers are authoritative for provider-side delivery/action outcomes, which must be reconciled into AXIS.

n8n may call approved APIs and transform data, but it must not keep a separate, conflicting copy of reminder/task state or directly execute high-impact actions without the backend gate.

6. Worker and scheduling design

- Persist due jobs or derive due work from reminder records.
- Claim work atomically (e.g. row locking / skip-locked pattern or equivalent) so multiple workers do not process the same item.
- Use leases/claim expiry so a crashed worker's job can be reclaimed.
- Record attempt ID, provider, idempotency key, timestamps, result, provider message ID, and error category.
- Use bounded retries and backoff; avoid infinite loops.
- A model outage must not prevent a reminder with a known message template from sending.
- Morning briefing generation can use the model, but the job itself and retry state are durable.
- After deploy/restart/restore, reconcile due and overdue jobs without flooding the user.
- Use a fake clock in tests to exercise schedules and expiry.

7. AI orchestration

- Use provider-neutral interfaces for generation, embeddings (if added), transcription, and speech.
- Validate structured outputs before domain writes.
- Give the model tools that call narrowly scoped domain services, not raw unrestricted database or shell access.
- Tool authorization is enforced server-side regardless of what the model says.
- Distinguish retrieval facts from model inference.
- Keep prompts/version identifiers and non-sensitive model metadata for debugging; avoid logging full private context by default.
- Provider fallback must respect model capabilities and privacy policy. Do not send sensitive data to another provider without the same allowed data-handling posture.
- Use bounded timeouts and token/cost budgets; do not retry endlessly.

8. Security boundaries

- Authentication and authorization are separate from model reasoning.
- Verify WhatsApp webhook signatures using provider guidance.
- Never trust sender phone number alone without binding it to the configured owner and validating the channel flow.
- Secrets live in deployment secret storage/environment variables, not the database as plaintext or source code.
- Apply least privilege and allow credentials to be revoked.
- Sanitize rendered content and defend against prompt injection from ingested data.
- Apply rate limits to login, capture, webhook and action endpoints.
- Use secure cookies/session strategy appropriate to selected auth approach.
- Redact secrets and sensitive payloads from logs.
- Enforce per-user ownership in every query; design future tenant isolation from day one.

9. Deployment and operations

Before choosing a host, verify:

- API process uptime and sleep behavior.
- Worker process support and scheduled execution guarantees.
- PostgreSQL backup/export availability and retention.
- TLS, secrets, log access, outbound provider connectivity.
- Webhook HTTPS reachability.
- Cost after free quota and behavior when quota is exhausted.
- Restore procedure and measured recovery time.

Keep staging/sandbox separate from production. A deployment is not complete until health checks, migrations, webhook setup, worker execution, backup, and restore procedures are tested.

10. Architecture decisions to record

Record each decision with date, context, chosen option, alternatives, consequences, and revisit trigger. Do not silently change database, auth, worker, provider, or approval architecture.

AXIS — Database and Memory Design

Version: 1.0 | Status: Logical schema; exact ORM syntax is implementation-time

1. General data rules

- PostgreSQL is the source of truth.
- Use UUID primary keys unless the chosen ORM has a strong reason not to.
- Persist timestamps in UTC (`timestamptz`); store the user's IANA timezone (e.g. `Asia/Kolkata`) on the user profile.
- Add `created_at` and `updated_at` to mutable domain records.
- Use foreign keys, indexes, constraints, and migrations.
- Scope user-owned records by `user_id` even in single-user MVP.
- Use transactions for operations that must commit together.
- Do not store secrets, provider tokens, or raw authentication credentials in memory records.
- Use soft deletion/archive only when required for user recovery/audit; honor deletion requests and retention policy.

2. Logical entities

users

- `id`
- `email` or provider subject, depending on auth design
- `display_name`
- `timezone`
- `quiet_hours_start`, `quiet_hours_end` (local wall-clock times; nullable until configured)
- `preferences_json`
- `created_at`, `updated_at`

projects

- `id`, `user_id`
- `name`, `description`
- `status` (`active`, `paused`, `archived`, `completed`)
- `priority` (nullable)
- `start_date`, `target_date` (nullable)
- `metadata_json`
- `created_at`, `updated_at`, `archived_at`

memory_items

- `id`, `user_id`, `project_id` nullable
- `kind` (preference, fact, decision, context, routine, relationship, other)
- `content` (text)
- `scope` (`global`, `project`)
- `truth_status` (`confirmed`, `assumed`, `suggested`)
- `confidence` (nullable numeric within 0–1)
- `source_type` (user, conversation, imported_file, integration, inferred)
- `source_ref` (nullable, sanitized reference)
- `sensitivity` (`ordinary`, `sensitive`)
- `save_consent_at` (nullable; required when policy says confirmation is needed)
- `valid_from`, `valid_until` (nullable)
- `archived_at`, `deleted_at` (nullable)
- `created_at`, `updated_at`, `last_accessed_at`

Indexes: (user_id, scope, project_id), status/archive fields, created time, full-text search vector. Consider pgvector only after retrieval evaluation.

tasks

- `id`, `user_id`, `project_id` nullable
- `title`, `description`
- `status` (`inbox`, `planned`, `in_progress`, `blocked`, `completed`, `cancelled`, `deferred`)
- `priority` (`low`, `normal`, `high`, `critical`)
- `due_at` nullable, `due_timezone` nullable
- `estimated_minutes` nullable
- `scheduled_start_at`, `scheduled_end_at` nullable
- `recurrence_rule` nullable
- `parent_task_id` nullable
- `source_channel` (`web`, `whatsapp`, `manual`, `integration`, `assistant`)
- `completion_at`, `archived_at` nullable
- `created_at`, `updated_at`

Constraints: scheduled end must be after start; estimate must be positive when present. Index (user_id, status, due_at) and project.

task_dependencies

- `id`, `user_id`, `task_id`, `depends_on_task_id`
- `dependency_type` (initially `blocks`)

- `created_at`
- Prevent self-dependency and duplicate edges; detect cycles at service layer.

task_events

- `id`, `user_id`, `task_id`
- `event_type`, `from_status`, `to_status`
- `metadata_json` with no unnecessary sensitive text
- `actor_type` (`user`, `assistant`, `system`)
- `created_at`

time_blocks

- `id`, `user_id`, `task_id` nullable, `plan_id` nullable
- `title`, `start_at`, `end_at`, `timezone`
- `kind` (`fixed_commitment`, `focus`, `task`, `break`, `buffer`, `personal`)
- `status` (`proposed`, `accepted`, `completed`, `skipped`, `cancelled`)
- `source` (`user`, `planner`, `calendar_read`)
- `created_at`, `updated_at`
- Do not persist overlapping accepted blocks unless explicitly allowed (e.g. a block may be informational); validate overlap in service logic.

plans

- `id`, `user_id`
- `period_start`, `period_end`
- `status` (`draft`, `proposed`, `accepted`, `superseded`, `archived`)
- `summary`, `constraints_json`
- `generated_by` (model/provider metadata, no prompt secrets)
- `created_at`, `updated_at`, `accepted_at`

reminders

- `id`, `user_id`
- `task_id` nullable, `memory_item_id` nullable, `plan_id` nullable
- `title`, `body_template` (minimize sensitive content in notifications)
- `due_at`, `timezone`
- `status` (`scheduled`, `processing`, `sent`, `delivered`, `failed`, `unknown`, `cancelled`, `snoozed`)
- `channel_policy_json` (WhatsApp primary, email fallback; explicit config)
- `fallback_after_at` nullable
- `next_attempt_at` nullable
- `last_attempt_at` nullable
- `idempotency_key` unique per logical notification
- `created_by`, `created_at`, `updated_at`, `cancelled_at`

notification_attempts

- `id`, `user_id`, `reminder_id`
- `channel` (`whatsapp`, `email`)
- `provider`
- `status` (`queued`, `sending`, `sent`, `delivered`, `failed`, `unknown`, `suppressed`)

- `attempt_number`
- `provider_message_id` nullable
- `idempotency_key`
- `started_at`, `finished_at` nullable
- `error_code`, `error_category`, `error_summary` (redacted)
- `provider_status_at` nullable
- `created_at`
- Unique constraint on provider/channel idempotency tuple where supported.

action_proposals

- `id`, `user_id`
- `action_type`
- `title`, `description`
- `destination_summary`
- `payload_json` (validated, immutable after proposal)
- `payload_hash`
- `risk_level` (`low`, `medium`, `high`, `critical`)
- `status` (`draft`, `pending_approval`, `approved`, `rejected`, `expired`, `executing`, `succeeded`, `failed`, `unknown`, `cancelled`)
- `expires_at` (default 24h after approval request)
- `approved_at`, `approved_by` nullable
- `executed_at`, `result_summary` nullable
- `idempotency_key`
- `created_at`, `updated_at`

approval_events

- `id`, `user_id`, `action_proposal_id`
- `event_type` (`requested`, `approved`, `rejected`, `expired`, `execution_started`, `execution_succeeded`, `execution_failed`, `execution_unknown`, `revoked`)
- `actor_type`, `actor_ref` nullable
- `payload_hash`
- `metadata_json` redacted
- `created_at`

integrations

- `id`, `user_id`
- `provider`, `integration_type`
- `status` (`disconnected`, `pending`, `connected`, `degraded`, `revoked`, `error`)
- `external_account_ref` nullable
- `scopes_json`
- `secret_reference` (reference to secret store, never raw secret)
- `last_checked_at`, `last_error_summary` nullable
- `created_at`, `updated_at`, `revoked_at`

audit_events

- `id`, `user_id`
- `event_type`, `actor_type`, `actor_ref` nullable

- `entity_type`, `entity_id` nullable
- `correlation_id`
- `outcome`
- `metadata_json` (allow-listed, redacted)
- `created_at`
- Append-only from application code; restrict update/delete permissions.

conversations and messages (optional for MVP)

- `conversations`: `id`, `user_id`, `channel`, `status`, timestamps.
- `messages`: `id`, `conversation_id`, `direction`, `provider_message_id`, `content` or encrypted/redacted representation, `received_at`, `processed_at`, `status`.
- Keep retention configurable. Do not treat message history as the only source of memory.

system_jobs / worker_leases (implementation choice)

A durable queue table or a reminder-driven claim strategy may be used. It must support atomic claiming, lease expiry, attempts, next retry, and deduplication. Document chosen approach.

3. Memory behavior

Save policy

- Routine/non-sensitive user-provided information: save by default when useful and relevant.
- Sensitive/personal information: ask before saving.
- Credentials/secrets: never save as ordinary memory.
- Inferred details: mark `assumed` or `suggested`; do not present as confirmed.
- Keep source/provenance and confidence where feasible.

Retrieval policy

- Search global memory plus relevant active project memory.
- Include archived project memory when relevant, clearly labeled archived.
- Filter deleted records.
- Prefer confirmed, current records over stale assumptions.
- When facts conflict, show the conflict and ask for correction rather than silently choosing.
- Do not infer that absence from memory means a fact is false.
- Retrieval should return source/updated time to the assistant so it can explain why it believes something.

Correction and deletion

- Correction should update the current record and preserve a minimal audit event without retaining deleted sensitive content.
- Deletion should prevent normal retrieval immediately. Any retention exceptions must be documented and limited.
- Offer a user-facing memory browser with search and filters by scope/type/status.
- Export is desirable but not an MVP release blocker.

4. Transactions and consistency

- Creating a task and its reminder should be transactional when both are confirmed in one operation.
- Approval status change and creation of execution job must be atomic.
- Worker claim and attempt record must prevent concurrent duplicate processing.
- Webhook updates must be idempotent.
- Task completion and recurring next-occurrence creation must be idempotent.

- Never rely only on in-process memory for pending jobs.

5. Migration and indexing checklist

- Include indexes for all frequent user-scoped list/search paths.
- Add database constraints for valid enum-like values and date ranges.
- Test migrations from an empty database and from the previous schema version.
- Seed only synthetic sample data.
- Do not put real personal memory or credentials in seed files.

AXIS — Security, Privacy, and Approval Policy

Version: 1.0 | Release-blocking policy

1. Security objectives

- Only the owner can access their records and connected accounts.
- External actions cannot be executed by prompt injection or by bypassing the UI.
- Credentials are revocable and stored outside ordinary application records where practical.
- Sensitive personal information is not saved to memory without consent.
- Security-sensitive behavior is auditable without logging unnecessary private content.
- System failure must not cause unsafe retries or duplicate consequential actions.

2. Trust boundaries

Treat all of the following as untrusted input:

- Model-generated arguments and summaries.
- Email, web pages, PDFs, imported files, calendar descriptions, and external API content.
- WhatsApp message content until sender/channel identity is verified.
- n8n workflow input/output and webhook payloads.
- Client-side form fields and query parameters.

Instructions found inside ingested content do not override system policies or user authorization. Never let an external document instruct AXIS to reveal secrets, change policies, or execute actions.

3. Authorization architecture

- Enforce authorization in backend domain services and API middleware.
- Every user-owned query is scoped to the authenticated user ID.
- The model can request a tool/action, but the backend decides whether it is permitted.
- Never expose generic “run arbitrary code/SQL/shell” tools to the assistant.
- External actions must pass through the central approval service.
- UI hiding/disabled controls are usability features, not security boundaries.

4. Approval lifecycle

1. **Draft:** proposal is being assembled; cannot execute.
2. **Pending approval:** immutable payload is shown to user; 24-hour expiry starts.

3. **Approved:** valid owner approval recorded; high-risk action also requires fresh web authentication.
4. **Executing:** execution job claimed once; approval is consumed according to action policy.
5. **Succeeded / failed / unknown:** terminal or reconciliation state.
6. **Rejected / expired / cancelled:** cannot execute.

Each proposal includes:

- Unique action ID.
- Action type and plain-language explanation.
- Exact destination/recipient/account.
- Exact content or effect preview.
- Immutable payload and cryptographic hash.
- Risk level.
- Expiration timestamp.
- Idempotency key.
- Audit history.

Required backend checks before execution

- Authenticated owner matches proposal owner.
- Proposal is approved and not expired/cancelled/rejected.
- Payload hash matches the approved payload.
- Destination and action type match the approved proposal.
- Risk policy is satisfied.
- Approval has not already been consumed.
- Idempotency key has not produced a completed or unresolved external effect.
- Required integration is connected and scopes are sufficient.

Any payload or destination change requires a new proposal and approval. An expired approval cannot be revived by a replayed webhook or message.

5. High-risk actions

High-risk actions require explicit approval plus fresh web authentication. Exact taxonomy and freshness window remain open decisions. Until finalized, classify uncertain actions conservatively. Potential high-risk examples include financial transactions, purchases, publishing publicly, deleting important external records, sending sensitive information, changing security settings, or submitting legally meaningful forms.

The implementation must support:

- A risk classifier/policy table owned by the backend.
- A “requires fresh authentication” flag on the proposal.
- A web authentication challenge before execution.
- Audit event for the challenge and result.
- No downgrade of risk level by model output alone.

Do not implement financial transaction execution in MVP.

6. WhatsApp approvals

- Verify provider webhook signatures.
- Bind the sender to the configured owner through an explicit setup/verification process.
- Do not treat an arbitrary “yes” message as approval for any pending action.

- Include a clear action identifier and concise summary in approval prompts.
- Require an unambiguous action-specific response.
- Handle duplicate/reordered webhooks idempotently.
- High-risk approval cannot be completed solely through WhatsApp; fresh web authentication is required.
- If message context is ambiguous, ask the user to open the dashboard.

7. Credentials and integrations

- Use environment variables or a managed secret store for API keys and session secrets.
- Store only a `secret_reference` in the application database where possible.
- Encrypt sensitive tokens at rest if they must be persisted; restrict decryption to the integration service.
- Do not log tokens, authorization headers, webhook signatures, full message bodies, or secret-bearing URLs.
- Support disconnect/revoke and document the steps to revoke provider-side access.
- Use least-privilege OAuth scopes; Calendar begins read-only.
- `.env.example` lists variable names with safe placeholder values only.
- Rotate secrets after suspected exposure; never commit them to fix local setup.

8. Memory privacy

- Routine information can be remembered by default when useful.
- Ask before saving sensitive/personal facts.
- Do not save passwords, access tokens, private keys, or financial credentials as memory.
- Provide memory inspection, correction, archive, and delete.
- Show whether information is confirmed, assumed, or suggested.
- Minimize sensitive content in reminders and notification previews.
- Logs and analytics should avoid storing memory text or message bodies unless strictly necessary and explicitly configured.

9. Web and model security

- Validate all structured model output against runtime schemas.
- Escape output in the UI and prevent script injection.
- Add rate limits and request-size limits.
- Use CSRF protection if cookie-based auth is selected.
- Use secure, HTTP-only cookies or a similarly robust session approach.
- Defend against prompt injection by separating system policy from retrieved content and treating retrieved content as data.
- Tool allowlists must be narrow and typed.
- Never allow the model to set its own authorization, risk level, user ID, or approval state.

10. Audit and incident handling

Record:

- Login/security events appropriate to auth provider.
- Memory save/correction/deletion (without copying sensitive deleted content).
- Approval requested/approved/rejected/expired.
- Action execution started and outcome.
- Reminder delivery/fallback failures.

- Integration connect/revoke.
- Backup and restore test outcomes.

Audit events should be append-only from normal application code, have correlation IDs, and use redacted metadata. Document an incident procedure: disable affected integration, revoke credentials, preserve relevant redacted logs, assess exposure, rotate secrets, restore if needed, and inform the owner.

11. Backup and recovery policy

- Automated database backup daily.
- Retain seven daily backups.
- Encrypt backups and restrict access.
- Exclude plaintext provider secrets from ordinary database backups; protect secret storage separately.
- Perform an isolated restore test monthly.
- Target RPO ≤24 hours and RTO ≤4 hours; verify against the selected provider and plan.
- After restoration, reconcile reminder due times, notification attempts, provider statuses, and approval/action states to avoid missed or duplicate effects.
- A successful backup job is not equivalent to a successful restore test.

12. Release-blocking security tests

- Direct API call cannot execute pending/unapproved action.
- Expired approval fails.
- Altered payload hash fails.
- Duplicate approval/replay executes no more than once.
- Concurrent workers cannot execute the same action twice.
- Cross-user access is denied.
- Invalid webhook signature is rejected.
- Secrets do not appear in logs, source maps, bundles, test snapshots, or error responses.
- High-risk action cannot proceed without fresh web authentication.
- Restore test passes and reminder reconciliation is documented.

AXIS — Reminder and Reliability Specification

Version: 1.0 | Priority: Highest

1. Design objective

A reminder is a durable obligation with observable delivery state. It must not exist only in a chat transcript, model context, browser timer, or n8n execution history.

2. Reminder lifecycle

Suggested state machine:

```
scheduled -> processing -> sent -> delivered
                    |          \-> unknown
                    |-> failed
unknown -> reconciled delivered / failed / retry scheduled
failed -> retry scheduled -> processing
scheduled -> cancelled
scheduled -> snoozed -> scheduled
```

Implementation may refine states, but must distinguish:

- Internal queueing from provider send.
- Provider accepted/sent from confirmed delivered.
- Confirmed failure from uncertain outcome.
- Cancellation from completion.

3. Reminder record requirements

Each reminder must include:

- Unique ID and user ID.
- Human-readable title and minimal notification body.
- Due timestamp in UTC and IANA timezone.
- Linked task/project/entity if relevant.
- Channel policy and fallback policy.
- Current status.
- Unique logical idempotency key.
- Attempt history.
- Next attempt/fallback timestamp.
- Created/cancelled timestamps.
- Correlation ID for tracing.

4. Creation policy

- If user provides a clear time/date, confirm interpretation and persist it.
- Interpret relative dates using the user's configured timezone.
- If time is missing, propose a time based on deadline and importance; ask before scheduling.
- If date/time is ambiguous, ask a single concise clarification.
- Before creating a reminder, show the title, due date/time with timezone, and channel policy.
- Support user edits, cancellation, snooze, and rescheduling.
- For deadlines, support configurable reminders at creation, before deadline, on due date, and after overdue. Exact offsets should be user-configurable; do not hard-code arbitrary timing without documenting it.
- Quiet hours and focus blocks apply to non-urgent notifications. Urgent exceptions must be explicit and configurable.

5. Worker processing

1. Find due reminders with `next_attempt_at <= now` or `due_at <= now`, status eligible.
2. Atomically claim the reminder using a lease/row-lock mechanism.
3. Check cancellation, quiet hours, user preferences, and idempotency.
4. Create a notification-attempt record before contacting the provider.
5. Call the provider with a stable idempotency key if supported.
6. Persist provider response and provider message ID.
7. Update status and next action based on provider semantics.
8. Release/expire claim safely.
9. Emit metrics and redacted structured logs.

A crash after provider send but before database update is a dangerous ambiguity. On restart, reconcile by provider message ID/idempotency key when possible. Do not blindly resend an uncertain notification.

6. WhatsApp primary and email fallback

Policy: WhatsApp is primary. Email fallback is attempted for every reminder whose WhatsApp delivery is not confirmed, according to a configurable timeout/retry policy.

- “Not confirmed” includes explicit failure and unresolved/unknown delivery after timeout.
- Use provider status callbacks/webhooks where available.
- The fallback timer starts from a defined event (proposed default: first WhatsApp attempt accepted by provider); record the chosen policy in configuration.
- Before fallback, query/reconcile current provider state if supported.
- If WhatsApp later reports delivered after email fallback, record both outcomes; do not erase history.
- Deduplicate email fallback by reminder ID and channel.
- Do not send email repeatedly for every retry cycle; one logical fallback notification should have its own idempotency key.
- Provider-specific delivery statuses vary. The adapter must map them into AXIS's normalized status without claiming more certainty than the provider offers.
- If the owner has no verified email, mark fallback blocked and surface a configuration alert rather than silently dropping it.

OPEN DECISION: exact fallback timeout and retry intervals. Implement configurable values and choose conservative documented defaults only after discussing them with the owner.

7. Retries and backoff

- Bounded retries; use exponential backoff with jitter for transient errors.
- Classify errors: transient, permanent, auth/configuration, rate limit, unknown.
- Permanent errors should not retry indefinitely.
- Respect provider retry-after hints.
- Add a maximum attempt count and dead-letter/needs-attention state.
- Alert the owner when retries are exhausted or configuration prevents fallback.
- Never retry a consequential external action with unknown outcome until reconciled. Reminder notifications may be retried only under the idempotency policy.

8. Webhook and duplicate handling

- Verify signatures.
- Store provider event ID and deduplicate.
- Make callback handling idempotent; the same callback may arrive multiple times or out of order.
- Use provider timestamps/status precedence rules where documented.
- Reject callbacks for unknown/unbound provider message IDs, but log a redacted diagnostic.
- Do not let a webhook directly authorize unrelated actions.

9. Scheduling semantics

- Store timestamps in UTC and user timezone separately.
- Daylight-saving behavior must be correct for users/timezones that observe DST; never assume every local day is 24 hours.
- For recurring reminders, persist recurrence rule and next occurrence.
- Generate next occurrence idempotently.
- Missed reminders after downtime should be reconciled: decide whether to send, summarize, or suppress stale notifications based on age and urgency. Make the policy configurable and visible.
- Use a fake clock in tests.

- Reminder processing must continue if all AI providers are down.

10. Quiet hours and focus blocks

- Quiet hours are configurable in user's local time.
- Focus blocks may suppress non-urgent interruptions but must not delete or lose the reminder.
- A suppressed reminder is rescheduled or surfaced in the next briefing according to policy.
- Urgent exception categories must be explicitly configured.
- Do not infer a medical, safety, or other urgent exception without a defined policy.

11. Monitoring metrics

- Due reminders not yet claimed.
- Queue/worker lag.
- Attempts per reminder.
- WhatsApp sent/delivered/failed/unknown rates.
- Email fallback rate and result.
- Time from due time to provider acceptance and delivery confirmation.
- Unknown outcomes older than threshold.
- Duplicate attempts prevented.
- Failed jobs and dead-letter count.
- Worker heartbeat and provider health.

12. Required scenarios

- API/model unavailable at reminder due time.
- Worker restarts before claim, after claim, after provider call, and after DB write.
- Provider timeout with unknown outcome.
- Duplicate/reordered webhook.
- WhatsApp explicit failure.
- WhatsApp remains unconfirmed and email fallback triggers.
- Email fallback provider unavailable.
- Missing/unverified email.
- Quiet hours/focus block.
- Reminder cancelled while queued.
- Recurring reminder completes once.
- Timezone/DST boundary.
- Database restore with overdue reminders.
- Two workers attempt to claim the same reminder.

No release until these tests pass or are explicitly classified as non-blocking with owner sign-off; core reminder-loss, duplicate, and fallback failures are always blockers.

AXIS — API and Integration Contracts

Version: 1.0 | Contract-first guidance; endpoint names are proposed

1. API conventions

- Base path: `/api/v1`.
- JSON request/response unless an integration requires another format.
- Runtime validation for every request.
- Authenticated user scope from session/token only; never trust a client-supplied `user_id`.
- Standard errors: `{ "error": { "code": "...", "message": "...", "requestId": "..." } }`.
- Use ISO 8601 timestamps; return UTC timestamps and timezone where local interpretation matters.
- Paginate list endpoints.
- Use idempotency keys for retryable create/execute operations.
- Avoid returning secrets or raw provider payloads.
- Version contracts; do not break webhooks silently.
- Generate OpenAPI documentation from implementation or maintain it alongside schemas.

2. Proposed endpoint groups

Health

- `GET /health/live` — process is alive.
- `GET /health/ready` — dependencies required for readiness are available.
- `GET /api/v1/system/status` — authenticated owner-facing status of worker, reminders, integrations, and recent failures.

Assistant and capture

- `POST /api/v1/assistant/messages` — submit a user message and receive a response plus structured proposed changes.
- `POST /api/v1/capture` — parse/capture task, memory, or commitment.
- `GET /api/v1/conversations`
- `GET /api/v1/conversations/:id/messages`

The assistant endpoint must not be a generic unrestricted tool executor. Domain mutations must go through validated services.

Memory

- `GET /api/v1/memories`
- `POST /api/v1/memories`
- `GET /api/v1/memories/:id`
- `PATCH /api/v1/memories/:id`
- `DELETE /api/v1/memories/:id`
- `POST /api/v1/memories/search`

Search results should include scope, truth status, source/updated time, and archive status. Sensitive memory creation must enforce consent policy.

Projects and tasks

- `GET|POST /api/v1/projects`
- `GET|PATCH|DELETE /api/v1/projects/:id`
- `GET|POST /api/v1/tasks`
- `GET|PATCH /api/v1/tasks/:id`
- `POST /api/v1/tasks/:id/complete`
- `POST /api/v1/tasks/:id/defer`
- `POST /api/v1/tasks/:id/reschedule`

- `GET /api/v1/tasks/:id/events`

Do not allow arbitrary status transitions that violate domain rules.

Reminders

- `GET|POST /api/v1/reminders`
- `GET|PATCH /api/v1/reminders/:id`
- `POST /api/v1/reminders/:id/cancel`
- `POST /api/v1/reminders/:id/snooze`
- `GET /api/v1/reminders/:id/attempts`

Reminder creation returns the persisted record and current delivery policy. A “scheduled” response means a durable record/job exists, not that the message has been sent.

Plans

- `GET /api/v1/plans/current`
- `POST /api/v1/plans/generate`
- `POST /api/v1/plans/:id/accept`
- `POST /api/v1/plans/:id/reject`
- `PATCH /api/v1/plans/:id`

Plan generation proposes time blocks; acceptance updates AXIS records. Any external calendar write still requires an action proposal and approval.

Approvals and actions

- `GET /api/v1/approvals`
- `GET /api/v1/approvals/:id`
- `POST /api/v1/approvals/:id/approve`
- `POST /api/v1/approvals/:id/reject`
- `POST /api/v1/actions/:id/execute` (internal/service protected; never public as an unguarded endpoint)
- `GET /api/v1/actions/:id/status`

Approval and execution endpoints must enforce the security policy in docs/04-security-and-approvals.md.

Integrations

- `GET /api/v1/integrations`
- `POST /api/v1/integrations/:provider/connect`
- `POST /api/v1/integrations/:provider/disconnect`
- `POST /api/v1/webhooks/whatsapp`
- `POST /api/v1/webhooks/email` if provider supports callbacks
- `POST /api/v1/webhooks/calendar` only if needed and securely verified

OAuth callback endpoints must validate state and redirect allowlists. Webhook endpoints must verify signatures and deduplicate events.

3. Error codes (minimum)

- `VALIDATION_ERROR`
- `UNAUTHENTICATED`
- `FORBIDDEN`
- `NOT_FOUND`

- `CONFLICT`
- `APPROVAL_REQUIRED`
- `APPROVAL_EXPIRED`
- `PAYLOAD_MISMATCH`
- `ACTION_ALREADY_CONSUMED`
- `INTEGRATION_DISCONNECTED`
- `PROVIDER_RATE_LIMIT`
- `PROVIDER_TIMEOUT_UNKNOWN`
- `RETRY_EXHAUSTED`
- `INTERNAL_ERROR`

Do not reveal internal stack traces or secrets in user-facing errors.

4. WhatsApp adapter

- Start in Twilio Sandbox/test configuration.
- Verify inbound webhook signatures.
- Bind the sandbox sender to the configured owner.
- Normalize inbound messages into a channel-neutral message contract.
- Store provider message IDs and status callbacks.
- Keep outbound send, delivery status, and approval parsing separate.
- Do not assume the Sandbox has the same capabilities or delivery guarantees as production WhatsApp.
- Avoid production outreach or sending to third parties without explicit owner authorization.
- Production onboarding, template approval, consent and region constraints must be validated before launch.

5. Email adapter

- Provider choice is open.
- Require a verified destination address and explicit owner configuration.
- Use for fallback when WhatsApp delivery is unconfirmed.
- Store provider message ID, accepted/sent status, delivery callback if available, and error category.
- Keep one logical fallback per reminder/channel unless a documented policy says otherwise.
- Do not send private reminder content in subject lines or previews unless appropriate.

6. Google Calendar

- Read-only first.
- Request least-privilege read scopes.
- Import fixed commitments as read-only calendar-derived blocks with provider references.
- Do not write/update/delete external calendar events in the initial configuration.
- If write capability is later enabled, every write is a proposed action requiring explicit approval and an immutable payload.
- Handle token revocation, sync errors, timezones, and duplicate events.

7. Model provider adapter contract

Create a provider-neutral interface for:

- Text generation.

- Structured extraction.
- Optional embeddings.
- Optional transcription/speech.

Requirements:

- Timeouts and bounded retries.
- Provider/model identifier in metadata.
- Capability negotiation; do not assume every provider supports structured output or tool calling.
- Schema validation after every structured response.
- Fallback only to an explicitly configured provider and compatible capability.
- Respect data-handling restrictions when changing providers.
- Avoid logging full prompts or private context by default.

8. n8n boundary

Use n8n for bounded integrations and workflow orchestration where it saves time. Call authenticated AXIS APIs. Do not store the canonical task/reminder/approval state solely in n8n. Do not allow an n8n workflow to bypass the approval gate. Store workflow versions and credentials securely; test replay behavior.

9. Contract tests

- Auth and owner scope.
- Validation failures.
- Pagination.
- Idempotency on create/execute.
- Webhook signature and duplicate events.
- Provider timeout/unknown outcome.
- Approval expiry and payload mismatch.
- Integration disconnect/revocation.
- No secret leakage in responses.

AXIS — UI/UX Specification

Version: 1.0 | Visual direction: Premium, futuristic, calm command center

1. Experience principles

- Dark mode by default; optional light mode.
- Premium futuristic/JARVIS-inspired without excessive glow, animation, or visual noise.
- Clear typography and generous spacing; information-dense but not overwhelming.
- Concise, natural, intelligent assistant voice with slight wit when appropriate.
- Status, deadlines, risk, and next action must be more prominent than decoration.
- Accessible keyboard navigation, contrast, focus indicators, reduced-motion support, and responsive layouts.
- Do not use color alone to communicate status.

2. Global shell

Desktop

- Left sidebar: Home, Tasks, Projects, Memory, Calendar/Plan, Inbox/Approvals, Integrations, Settings.
- Main workspace: current section.
- Persistent assistant panel or expandable assistant surface.
- Top-level quick capture always accessible.
- Notification/system health indicators.
- Clear connected-account status and profile/settings access.

Mobile

- Chat-first default with a simple bottom or compact navigation.
- Quick capture available immediately.
- Easy access to today's agenda, reminders, approvals, and tasks.
- Avoid desktop-style three-column layouts on narrow screens.
- Keep approval summaries readable and require deliberate confirmation.

3. Dashboard / Home

Required cards/sections:

1. **Today:** fixed commitments, due tasks, reminder status.
2. **Top priorities:** limited, ranked list with reason.
3. **Deadlines:** upcoming and overdue commitments.
4. **Pending approvals:** count and clear next action.
5. **Daily briefing:** concise summary, risks, suggested focus.
6. **Plan:** today's time blocks with rest and buffers.
7. **Projects:** active project health and next milestones.
8. **Inbox / delivery health:** failed or unknown reminders, integration issues.
9. **Quick capture:** task, memory, reminder, or general note.
10. **Recent memory/activity:** small, searchable trail.

Allow hiding/reordering lower-priority widgets later; do not make customization a prerequisite for MVP.

4. Tasks

- Views: Today, Upcoming, Overdue, Inbox, Completed, project filter.
- Show status, priority, due date/time, project, estimated effort, dependencies.
- Actions: complete, edit, defer, reschedule, archive.
- Make overdue items actionable without shaming the user.
- When rescheduling a missed task, show a proposal and ask before changing the plan.
- Recurring task completion should clearly show the next occurrence.

5. Memory

- Searchable list with filters for global/project, type, confirmed/assumed/suggested, active/archived.
- Memory detail shows content, provenance/source, last updated, confidence/status, and project scope.
- Actions: correct, archive, delete.
- For sensitive facts, display consent state and avoid accidental exposure in notifications.
- Deletion confirmation should be proportionate; do not make correction/deletion difficult.

6. Inbox and approvals

- Separate actionable approvals from informational alerts.
- Each approval card shows action, exact recipient/destination, concise preview, risk, expiry, and consequences.
- Actions: approve, reject, inspect details.
- High-risk action requires web re-authentication at approval/execution boundary.
- Do not use vague buttons such as “OK” for consequential actions; use explicit labels such as “Approve sending email.”
- Display expired proposals as expired and require a fresh proposal.
- Show execution result as succeeded, failed, or unknown; never imply success while unknown.

7. Reminder detail

Show:

- Title and linked task/project.
- Due date/time and timezone.
- Channel policy.
- Status and delivery attempts.
- WhatsApp provider status and email fallback status.
- Last attempt, next retry, error summary (redacted).
- Edit, snooze, cancel actions.

Use plain-language status labels such as “Sent; delivery not confirmed” rather than misleading “Delivered.”

8. Assistant interaction

- Keep responses concise by default.
- Distinguish facts, assumptions, and suggestions.
- For proposed changes, show a compact structured summary before confirmation.
- Ask a single direct question when one essential detail is missing.
- Never claim a record was saved/scheduled until backend confirms persistence.
- Explain uncertainty and next step, not internal stack traces.
- Provide undo/cancel where safe and technically possible.
- Assistant should not create clutter by narrating every internal step.

9. Planning/calendar

- Day and week views.
- Fixed commitments visually distinct from flexible tasks and breaks.
- Show buffers and overload warnings.
- Plans are proposals until accepted.
- Calendar imported in read-only mode initially.
- If a plan cannot fit, explain the conflict and offer options rather than overbooking.
- Timezone must be visible when ambiguity is possible.

10. Integrations/settings

- Connection status, last successful check, permissions/scopes, disconnect/revoke.

- Clear environment indicator for sandbox vs production.
- Never display full secrets after saving.
- Configure timezone, quiet hours, notification channels, email fallback, briefing time, and model/provider budget.
- Explain which data each integration can read/write.

11. Visual tokens (initial guidance)

The coding agent may refine these after a small prototype:

- Base: near-black/charcoal background, layered neutral surfaces.
- Accent: restrained electric blue or cool cyan.
- Secondary accents: muted slate; reserve amber/red for attention/error.
- Corners: moderately rounded, not pill-shaped everywhere.
- Motion: subtle transitions; no distracting constant animations.
- Use clear type scale and consistent spacing.
- Avoid excessive gradients, neon borders, and sci-fi clichés.

12. Accessibility and responsive acceptance

- Keyboard can reach all actions and dialogs.
- Focus is visible.
- Contrast is sufficient.
- Screen reader labels exist for icon-only actions.
- Dialogs manage focus correctly.
- Layout works at narrow mobile widths and desktop widths.
- Reduced motion preference is respected.
- Approval actions are not dangerously adjacent without confirmation.

AXIS — Implementation Roadmap

Version: 1.0 | Rule: Complete one milestone and verify it before moving on.

Milestone M0 — Foundation and decisions

Goal: establish a safe, runnable project without building product features prematurely.

Tasks:

- Inspect repository and existing assets before changes.
- Confirm package manager, monorepo approach, auth approach, database/ORM, hosting constraints, and model-provider options; record decisions.
- Create repo structure, lint/format/typecheck/test scripts.
- Add `.env.example` with variable names only.
- Set up local PostgreSQL and migration workflow.
- Add health endpoint and structured logging with request/correlation IDs.
- Add CI checks where available.
- Add threat model and initial runbook.

Acceptance:

- Clean install and documented local startup.
- Empty database can be migrated from scratch.
- Health checks work.
- No secrets committed.
- Lint, typecheck, and test commands run successfully.
- No paid service is provisioned without owner approval.

Stop and report before M1 if any major decision is blocked.

Milestone M1 — Identity, projects, memory, tasks

Goal: persistent core records with a minimal usable web UI.

Tasks:

- Secure single-owner authentication and server-side ownership scope.
- Database migrations for users, projects, memory, tasks, task events.
- CRUD APIs with validation and pagination.
- Memory search using PostgreSQL full-text search initially.
- Memory inspect/correct/archive/delete and sensitive-memory consent.
- Task capture/list/edit/complete/defer.
- Minimal responsive dashboard.
- Seed synthetic demo data only.

Acceptance:

- Records survive restart.
- User scope tests pass.
- Memory uncertainty/provenance visible.
- Sensitive-memory consent enforced.
- Migrations and API contract tests pass.

Milestone M2 — Durable reminder engine

Goal: dependable reminders independent of AI and web chat.

Tasks:

- Reminder and notification-attempt schema.
- Worker process with atomic claims/leases.
- Fake-clock test harness.
- Retry classification/backoff, cancellation, snooze, quiet hours.
- Structured attempt status and owner-facing system status.
- Email/WhatsApp adapter interfaces using mocks first.
- Reconciliation for unknown provider outcomes.

Acceptance:

- Worker restart does not lose reminders.
- Concurrent workers do not duplicate a logical attempt.
- Provider timeout is represented as unknown and reconciled.
- Model outage does not stop scheduled reminders.

- Core reliability scenarios pass.

Milestone M3 — WhatsApp Sandbox and email fallback

Goal: end-to-end messaging in a controlled test environment.

Tasks:

- Twilio Sandbox setup using owner-provided credentials.
- Verify webhook signatures and owner binding.
- Inbound message normalization.
- Outbound reminder send and provider status callbacks.
- Email adapter and verified owner destination.
- Fallback timeout/retry configuration.
- Idempotent callbacks and one logical email fallback per reminder.
- Clear sandbox/test labels in UI.

Acceptance:

- Test reminder sent to authorized sandbox recipient.
- Delivery states accurately reflect provider semantics.
- Unconfirmed WhatsApp triggers email fallback under configured policy.
- Replayed webhooks do not create duplicate fallback.
- Missing email/configuration surfaces an alert.
- No real third-party messaging without owner approval.

Milestone M4 — Approval/security gate

Goal: central, testable authorization for all external actions.

Tasks:

- Immutable action proposal and payload hash.
- 24-hour expiry.
- Approve/reject via web.
- Verified WhatsApp approval for non-high-risk actions, if safe and feasible.
- High-risk web re-auth gate (risk taxonomy/freshness policy must be decided).
- Idempotent execution, audit trail, reconciliation for unknown outcomes.
- Penetration-style tests of direct API bypasses.

Acceptance:

- Unapproved, expired, altered, replayed, and concurrent execution attempts fail safely.
- High-risk action requires fresh web authentication.
- Audit events cover full lifecycle.
- No live external action is enabled by default.

Milestone M5 — Dashboard and daily experience

Goal: make the core system useful every day.

Tasks:

- Home command center.

- Today/upcoming/overdue task views.
- Reminder delivery detail and inbox.
- Memory browser.
- Pending approvals.
- Daily briefing generation and delivery.
- Weekly plan proposals with constraints, rest and buffers.
- Missed-task detection and rescheduling proposal.

Acceptance:

- Briefing reflects persisted state.
- Planner avoids overlapping accepted blocks and fixed commitments.
- Missed tasks are proposed for rescheduling, not silently moved.
- Briefing failure does not break reminders.

Milestone M6 — Calendar read-only and operational hardening

Goal: integrate schedule context and prove recovery.

Tasks:

- Google Calendar read-only integration with least-privilege scopes.
- Integration disconnect/revocation.
- Daily backups, seven-day retention, monthly restore test procedure.
- Runbook for provider failure, worker backlog, secret rotation, restore.
- Verify RPO/RTO targets against actual hosting plan.
- End-to-end tests for recovery and reminder reconciliation.

Acceptance:

- Calendar events appear as read-only commitments.
- Revoked access is detected and handled.
- Restore test is documented and successful.
- Reminder/action reconciliation after restore prevents duplicate effects.

Milestone M7 — Follow-on capabilities

Only after core release gates pass:

- Voice-note transcription and speech responses.
- Rich client/project pipeline.
- CAT/fitness tracking and other domain-specific modules.
- Browser automation behind narrow allowlists and approval gates.
- Additional integrations and model providers.
- Production WhatsApp setup after provider eligibility/consent review.
- Multi-user tenancy, onboarding, billing, and commercial productization.

Milestone reporting template

For each milestone, report:

- Scope completed.
- Files/modules changed.

- Commands actually run and results.
- Acceptance criteria pass/fail.
- Manual setup still needed.
- Risks/open decisions.
- Estimated next step.
- Whether any external service, paid resource, or real-world action was touched.

AXIS — Test Plan and Acceptance Criteria

Version: 1.0 | Release gate: tests must prove safety and reliability, not just happy paths

1. Test layers

- Unit tests: pure domain rules, state transitions, date/time, risk policy, recurrence.
- Integration tests: PostgreSQL transactions, worker claims, repository ownership scopes.
- API/contract tests: validation, auth, error contracts, idempotency, webhook signatures.
- Provider adapter tests: mocked provider success/failure/timeout and status mapping.
- End-to-end tests: capture → persistence → reminder → status; proposal → approval → action result.
- Security tests: authorization bypass, prompt injection, secret leakage, replay/concurrency.
- Recovery tests: backup restore and post-restore reconciliation.
- Manual sandbox tests: WhatsApp Sandbox and email provider setup.

2. Release-blocking scenarios

Reminder durability

1. Create reminder, restart API and worker, verify reminder remains due and is processed.
2. Disable model provider at due time; reminder still processes.
3. Crash worker before provider call; lease expires and work is reclaimed.
4. Simulate provider timeout after request may have reached provider; status becomes unknown and reconciliation runs before retry.
5. Send duplicate/out-of-order status callbacks; one logical state transition and no duplicate fallback.
6. WhatsApp explicit failure triggers configured fallback/retry.
7. WhatsApp delivery unconfirmed past configured timeout triggers email fallback.
8. Email unavailable or unverified: fallback blocked status and owner alert appear.
9. Cancel reminder while queued; it is not sent after cancellation wins the race.
10. Two workers claim same reminder concurrently; only one logical send occurs.
11. Restore database with overdue reminders; reconciliation does not flood or duplicate notifications.
12. Test quiet hours, focus block, timezone conversion, DST where relevant, and recurrence.

Approval gate

1. Direct API execution without approval returns `APPROVAL_REQUIRED`.
2. Expired approval returns `APPROVAL_EXPIRED`.
3. Modified payload hash returns `PAYLOAD_MISMATCH`.
4. Wrong user/owner cannot approve or execute.
5. Replayed approval webhook does not execute twice.

6. Two concurrent execute requests result in at most one effect.
7. High-risk action without fresh web authentication fails.
8. An approved action with changed destination requires new approval.
9. Unknown execution outcome is reconciled before retry.
10. Audit trail records request, approval/rejection/expiry, execution attempt, and outcome.

Memory and privacy

1. Routine fact can be saved under default policy.
2. Sensitive fact is not saved until consent.
3. Confirmed/assumed/suggested labels persist.
4. Correction updates current retrieval.
5. Deleted memory is excluded from normal search.
6. Archived project memory remains searchable when requested/relevant.
7. Secrets cannot be stored as ordinary memory.
8. Cross-user access is denied by service and repository tests.

Backup and recovery

1. Daily backup job creates a restorable backup.
2. Retention removes backups older than seven daily copies according to policy.
3. Restore into isolated environment succeeds.
4. RPO/RTO measurement is documented.
5. After restore, reminders, attempts, approvals, and uncertain actions reconcile safely.
6. Secrets are handled separately from ordinary database backups.

3. Acceptance by milestone

M0

• Clean install; formatting/lint/typecheck/test commands; migration from empty DB; health checks; no secrets.

M1

• Authenticated CRUD, ownership checks, memory consent, searchable memory, persistent tasks, audit/history where required.

M2

• Independent worker, durable reminder state, retry/backoff, idempotency, cancellation, unknown-outcome reconciliation.

M3

• Sandbox messaging, signature validation, normalized delivery state, email fallback, no duplicate sends in replay tests.

M4

• Server-side approval gate, 24-hour expiry, immutable payload, high-risk re-auth, complete audit lifecycle.

M5

• Dashboard displays persisted truth, daily briefing and plan proposals are accurate and non-overlapping, reschedule proposals require confirmation.

M6

• Read-only calendar, backups, successful restore test, documented operations runbook.

4. Reliability targets

These are targets to measure, not claims to make before measurement:

- No known silent loss of critical reminders.
- Zero approval-gate bypasses in the release test suite.
- Zero duplicate consequential effects in replay/concurrency tests.
- Successful monthly restore test.
- RPO target ≤24 hours and RTO target ≤4 hours, validated against hosting.
- Every reminder has an observable final or unresolved state; unresolved items are surfaced.

Do not promise 100% delivery because external providers, connectivity, quotas, and account eligibility are outside AXIS's control.

5. Test data and privacy

- Use synthetic data in automated tests.
- Never use real personal memory or secrets in fixtures.
- Redact message bodies and provider responses in snapshots.
- Keep sandbox and production credentials separate.
- Use deterministic fake clocks for expiry and scheduling tests.
- Do not send test messages to real third parties.

AXIS — Architecture Decision Log

Record material choices here. Do not silently replace decisions.

ADR-001 — Product form

- **Status:** Accepted
- **Decision:** AXIS is a personal AI operating system plus autonomous digital employee, initially single-user with future multi-user isolation in mind.
- **Reason:** Reduce cognitive load and coordinate memory, tasks, reminders, plans, and integrations.
- **Consequence:** Build a secure single-user MVP without hard-coding user identity into domain models.

ADR-002 — Modular monolith first

- **Status:** Accepted as initial direction
- **Decision:** Use a modular monolith for API/domain code and a separate worker process.
- **Reason:** Keeps deployment and maintenance manageable while protecting scheduler independence.
- **Revisit when:** Real scaling, availability, or isolation evidence justifies service separation.

ADR-003 — Database is source of truth

- **Status:** Accepted
- **Decision:** PostgreSQL is authoritative for tasks, reminders, approvals, plans, delivery attempts, integrations metadata, and audit events.
- **Reason:** Chat and model context are not durable or transactionally reliable.
- **Consequence:** n8n and model context cannot become competing sources of truth.

ADR-004 — Approval-gated external actions

- **Status:** Accepted
- **Decision:** All external-state-changing actions require explicit approval; high-risk actions also require fresh web authentication.
- **Reason:** The user wants AXIS to suggest actions but wait for approval.
- **Consequence:** Central backend gate, immutable payload hash, expiry, idempotency, audit and reconciliation are mandatory.

ADR-005 — Reminder delivery policy

- **Status:** Accepted
- **Decision:** WhatsApp is primary; email fallback is attempted for every reminder whose WhatsApp delivery is not confirmed, under a configurable timeout/retry policy.
- **Reason:** Reminders are the highest-trust product function.
- **Consequence:** Provider status callbacks, unknown outcome, fallback deduplication and operational alerts are required.

ADR-006 — WhatsApp Sandbox first

- **Status:** Accepted
- **Decision:** Begin with WhatsApp Sandbox; production sender/onboarding is deferred.
- **Reason:** Low-cost controlled testing before production messaging.
- **Consequence:** Clearly label sandbox mode; validate provider capabilities before launch.

ADR-007 — Calendar read-only first

- **Status:** Accepted
- **Decision:** Calendar integration starts read-only. Calendar writes require explicit approval and are not part of the initial baseline.
- **Reason:** Avoid accidental external schedule changes.
- **Consequence:** Imported events are context, not editable AXIS records.

ADR-008 — Memory privacy and truth status

- **Status:** Accepted
- **Decision:** Remember routine facts by default; ask before saving sensitive/personal facts. Label facts confirmed, assumed, or suggested.
- **Reason:** Useful memory must remain transparent and user-controlled.
- **Consequence:** Memory items require provenance, scope, status, and inspect/correct/delete UX.

ADR-009 — Backups and recovery

- **Status:** Accepted target policy
- **Decision:** Daily backups, seven daily copies, monthly isolated restore test, target RPO 24 hours and RTO 4 hours.
- **Reason:** Reminder and commitment records must be recoverable.
- **Consequence:** Validate actual hosting capabilities and reconcile due jobs after restore.

ADR-010 — Proposed technology defaults

- **Status:** Proposed; confirm in M0 before committing to hosting or paid providers
- **Decision:** React + TypeScript + Vite; Node.js + TypeScript + Express; PostgreSQL; provider adapters; separate worker; n8n for bounded workflows where useful.
- **Reason:** Familiar, practical stack and low operational complexity.
- **Consequence:** Keep integrations and AI providers replaceable.

Open decisions — resolve before the relevant milestone

1. Auth library/provider and session model.
2. ORM/query builder and monorepo/package manager.
3. Hosting provider, worker uptime, database backup and restore capabilities.
4. Model providers, fallback order, cost limits, privacy compatibility.
5. Email provider, sender domain, verification and fallback timeout/retries.
6. Exact high-risk action taxonomy and web-auth freshness window.
7. Approval revocation behavior once execution begins.
8. Quiet hours, focus-block behavior and urgent exception categories.
9. Whether PostgreSQL full-text search is sufficient or pgvector is justified.
10. Exact briefing time and reminder offsets.
11. Production WhatsApp provider/sender and launch requirements.
12. Calendar OAuth scopes and token storage approach.

Decision record template

ADR-XXX — Title

- **Status:** Proposed / Accepted / Superseded
- **Date:**
- **Context:**
- **Options considered:**
- **Decision:**
- **Consequences:**
- **Revisit trigger:**